Bridging Critical Reproduction Gaps in Superconducting Qubit Calibration

Proposed Solutions for the Four Critical Gaps in Google Quantum AI's Published Calibration Stack

Compiled March 15, 2026

Abstract. Our companion survey identified four *critical* reproduction gaps in Google Quantum AI's calibration stack: (1) the complete Optimus calibration DAG, (3) the Snake optimizer's inner-loop algorithm, (5) the TLS forecasting algorithm, and (10) the custom control electronics specifications. Here we present detailed, independently reconstructible solutions for each gap. For Gap 1, we synthesize calibration workflows from Qibocal, QUAlibrate, Qiskit Experiments, and Zurich Instruments LabOne Q to reconstruct a complete 50+ node DAG with concrete tolerances and timeout values. For Gap 3, we reverse-engineer the Snake's error model, propose three candidate inner-loop optimizers with pseudocode, and reconstruct the 16-weight training procedure. For Gap 5, we develop six TLS forecasting algorithms ranging from simple change-point extrapolation to FPGA-based real-time Bayesian tracking, grounded in newly measured TLS spectral diffusion constants. For Gap 10, we benchmark five commercial and one open-source control electronics platforms against Willow-level specifications and provide a complete wiring/filtering scheme. We find that all four gaps can be bridged with existing public knowledge, though significant experimental validation remains necessary.

Contents

I. Introduction . . . 3
II. Gap 1: Reconstructing the Complete Calibration DAG . . . 3
  II.A. Available Sources . . . 3
  II.B. Reconstructed DAG: Complete Node Specification . . . 3
  II.C. Drift Timescales for Timeout Values . . . 6
  II.D. Complete DAG Dependency Graph . . . 7
III. Gap 3: Reconstructing the Snake Optimizer . . . 7
  III.A. Published Details . . . 7
  III.B. Reconstructed Error Model . . . 7
  III.C. Proposed Inner-Loop Optimizers . . . 9
  III.D. Frequency Collision Rules . . . 9
  III.E. Reconstructed Pseudocode . . . 10
IV. Gap 5: TLS Forecasting Algorithms . . . 11

IV.A. The TLS Problem . . . 11
IV.B. Key Physical Parameters . . . 11
IV.C. Proposed Algorithms . . . 11
IV.D. Comparison of Algorithms . . . 13
IV.E. Fundamental Limitations . . . 13
V. Gap 10: Control Electronics Stack . . . 14
V.A. Target Specifications (Willow) . . . 14
V.B. Minimum Electronics Requirements . . . 14
V.C. Commercial Platform Comparison . . . 15
V.D. Wiring and Filtering Scheme . . . 16
V.E. Scaling Beyond 100 Qubits . . . 16
VI. Synthesis and Recommendations . . . 17
VI.A. Feasibility Assessment . . . 17
VI.B. Recommended Reproduction Roadmap . . . 17
References . . . 18

I. Introduction

This document addresses the four gaps assessed as *Critical* in our companion survey on superconducting qubit calibration:

Gap	Description	Section
1	Complete Optimus calibration DAG: node names, analysis functions, tolerances, timeouts	Section II
3	Snake optimizer inner-loop algorithm, frequency-separation rules, weight-training procedure	Section III
5	TLS forecasting algorithm: inputs, prediction method, horizon, accuracy	Section IV
10	Custom control electronics: DAC specs, signal generation, wiring, filtering	Section V

Table 1: The four critical reproduction gaps addressed in this document.

For each gap, we (a) review all publicly available information, (b) synthesize solutions from multiple independent sources, (c) propose concrete implementations with sufficient detail for reproduction, and (d) identify remaining uncertainties.

II. Gap 1: Reconstructing the Complete Calibration DAG

II.A. Available Sources

The Optimus framework [1] defines calibration as a DAG traversal problem but never publishes the complete node list. We reconstruct the full DAG by synthesizing five independent sources:

- **Qibocal** [2]: 37 protocol routines for transmon calibration (open-source).
- **QUAlibrate** [3]: Component-based DAG framework from Quantum Machines; demonstrated full multi-qubit tune-up in 140 seconds.
- **Qiskit Experiments** [4]: IBM's calibration library with prescribed ordering.
- **LabOne Q** [5]: Zurich Instruments' DAG-based bring-up; characterized 26/28 qubits across 7 chips in ~ 3.5 hours.
- **Google Patent US9940212B2** [6]: Adds that each node has two experiment tiers—a low-cost `check_data` and high-cost `calibrate`.

II.B. Reconstructed DAG: Complete Node Specification

We organize the DAG into 12 layers, totaling 52 nodes. Each node specifies: parameters calibrated, scan type, analysis function, tolerance, drift timescale (timeout), and dependencies.

II.B.1. Layer 0: System Prerequisites

Node	Parameters	Analysis	Tolerance	Timeout
Cryostat cooldown	Base temperature	Thermometer	< 15 mK	N/A
TWPA calibration	Pump freq, power, gain	S_{21} vs. freq	Gain flat across band	~ 24 h

Table 2: Layer 0: System-level prerequisites.

II.B.2. Layer 1: Readout Resonator Characterization

Node	Parameters	Analysis	Tolerance	Timeout
Resonator spec. (high power)	$f_{r,\text{bare}}$	Lorentzian fit to S_{21}	< 100 kHz	$\sim$ hours
Resonator punchout	$P_{\text{ro,opt}}$	2D bifurcation detection	Below bifurcation	$\sim$ hours
Resonator spec. (low power)	$f_{r,\text{dressed}}$, χ	Lorentzian fit	< 50 kHz	$\sim$ hours
Dispersive shift	$\chi = f_{r\|0\angle} - f_{r\|1\angle}$	Two-peak separation	$\chi > \kappa$	$\sim$ hours

Table 3: Layer 1: Readout resonator characterization. All nodes are fully parallelizable across the chip via frequency-multiplexed readout.

II.B.3. Layer 2: Qubit Spectroscopy

Node	Parameters	Analysis	Tolerance	Timeout
Two-tone spec.	f_{01}, α	Lorentzian peak	< 500 kHz	~ 1 h
e-f spec.	f_{12}, α	Peak detection	< 500 kHz	~ 1 h
Flux spec.	Sweet-spot bias, $f(\Phi)$	Transmon model fit	1 mΦ_0	~ 1 h

Table 4: Layer 2: Qubit identification and spectroscopy.

II.B.4. Layer 3: Readout Optimization

Six nodes: Time-of-flight $\rightarrow$ Readout frequency optimization $\rightarrow$ Readout amplitude optimization (Nelder-Mead, ~ 100 ms [7]) $\rightarrow$ Integration weights (matched filter) $\rightarrow$ Single-shot classification (Gaussian mixture / LDA in IQ plane) $\rightarrow$ Readout mitigation matrix.

Target: assignment fidelity $> 99\%$ (state-of-art: 99.5–99.77% at 56–140 ns integration [8]).

II.B.5. Layer 4: Single-Qubit Gate Calibration

Node	Parameters	Analysis	Tolerance	Timeout
Rabi amplitude	A_π, $A_{\pi/2}$	Sinusoidal fit	$< 1\%$ of π	~ 30 min
Ramsey	f_{01} (fine), T_2^*	Damped sinusoid	< 10 kHz	$\sim$ min–hours
DRAG	β	Cosine fit (error amp.)	$< 5\%$ of $-1/(2\alpha)$	Stable
Fine amplitude	Corrected A_π	Linear fit ($d\theta$ vs. N)	$< 10^{-4}$ rad	~ 30 min
Fine DRAG	Refined β	Error amplification	Leakage $< 10^{-4}$	Stable
AllXY	Verification	21-pair deviations	All within spec	N/A
Virtual Z	Software phases	Frame tracking	Exact	Stable

Table 5: Layer 4: Single-qubit gate calibration. The key tolerance from the Optimus patent is 10^{-4} radians for the π-pulse rotation angle.

II.B.6. Layer 5: Coherence Characterization

Five nodes: T_1 (exponential decay fit; fluctuates on sub-second to hour timescales [9]), T_2^* (Ramsey), $T_{2,\text{echo}}$ (Hahn echo), $T_{2,\text{CPMG}}$ (dynamical decoupling), T_φ (derived: $1/T_\varphi = 1/T_2 - 1/(2T_1)$).

II.B.7. Layer 6: Single-Qubit Benchmarking

Standard RB (exponential decay of Clifford fidelity; target EPC $< 10^{-3}$), Interleaved RB (gate-specific error), Half-angle calibration.

II.B.8. Layer 7: Flux Calibration

Node	Parameters	Analysis	Tolerance	Timeout
Flux bias	DC sweet-spot bias	Ramsey vs. flux	$5 \times 10^{-4}\ \Phi_0$	$\sim$ hours
Flux crosstalk matrix	M_{ij}	Linear regression	$< 10^{-3}$ of applied flux	$\sim$ hours
Cryoscope (FIR)	120 FIR coefficients	Regularized optimization	$< 0.1\%$ freq. error	$\sim$ hours
IIR distortion	4 exponential taus	Sum-of-exponentials fit	< 2 MHz residual	$\sim$ hours

Table 6: Layer 7: Flux calibration nodes. The cryoscope measures flux-pulse distortion at AWG sampling resolution (0.42 ns) [10].

II.B.9. Layer 8: Tunable Coupler Calibration

Five nodes in sequence: Anticrossing localization → Energy spectrum mapping (via adiabatic SWAP, requiring ≥ 50 ns rise/fall [11]) → Coupler flux distortion → Dispersive shift tuning → Idle-point calibration (target: $ZZ/2\pi < 10$ kHz).

II.B.10. Layer 9: Two-Qubit Gate Calibration

For CZ gates (Google architecture): Chevron scan (2D amplitude × duration) → Conditional phase calibration ($|\varphi_{\mathrm{CZ}} - \pi| < 0.01$ rad) → Leakage calibration (target $< 0.11\%$ per gate) → Single-qubit phase corrections (virtual Z).

II.B.11. Layers 10–11: Two-Qubit Benchmarking and System Verification

Two-qubit RB (target $< 0.6\%$ error), interleaved RB, layer fidelity, quantum volume, active reset calibration.

II.C. Drift Timescales for Timeout Values

Parameter	**Drift Timescale**	**Recommended Timeout**
T_1	Sub-second telegraphic (up to 10 Hz); hours slow	Minutes (ideal: ms)
Qubit frequency	± 25 kHz / 6 h; $1/f$ noise	1–4 hours
π-pulse amplitude	Allan deviation at ~ 1000 s	30 min
Readout parameters	Hours (amplifier drift)	2–4 hours
DRAG β	Stable (α fixed by junction)	Days–weeks
Flux bias	Minutes to hours	1–2 hours
Two-qubit gate	Hours (coupler drift)	1–2 hours

Table 7: Recommended timeout values based on measured drift timescales from the literature. Fast FPGA-based calibration can achieve ~ 31 ms combined frequency + amplitude recalibration [7].

II.D. Complete DAG Dependency Graph

```
CRYOSTAT → TWPA → RES_SPEC_HI → RES_PUNCHOUT → RES_SPEC_LO → DISPERSIVE_SHIFT
                                                  |
                                                  ▼
                  QUBIT_SPEC → FLUX_SPEC → FLUX_BIAS → FLUX_XTALK → CRYOSCOPE/IIR
                      |                                   |
                      ▼                                   ▼
            TIME_OF_FLIGHT → RO_FREQ → RO_AMP → INT_WGTS → SINGLE_SHOT → RO_MITIG
                      |
                      ▼
                 RABI_AMP → RAMSEY → DRAG → FINE_AMP → FINE_DRAG → ALLXY
                      |         |
                      ▼         ▼
                  T1/T2/T2E  VIRTUAL_Z
                      |
                      ▼
                 SQ_RB → INTERLEAVED_RB
                              |
      COUPLER_ANTICROSS → COUPLER_SPECTRUM → COUPLER_DISTORT → COUPLER_IDLE
                                                                  |
                                                                  ▼
                                          CHEVRON → CZ_PHASE → CZ_LEAK → CZ_Z_CORR
                                                                                |
                                                                                ▼
                                                      TQ_RB → LAYER_FIDELITY → QV
```

Listing 1: Complete reconstructed calibration DAG. Arrows indicate dependencies. Nodes at the same depth can be parallelized across qubits/pairs.

III. Gap 3: Reconstructing the Snake Optimizer

III.A. Published Details

The Snake optimizer [12] decomposes the $\sim 3N$-dimensional frequency configuration problem into local subproblems of dimension S^2 (scope parameter S). The paper publishes:

- The *outer loop*: graph traversal, scope parameter, constraint propagation.
- The *error model equation*: $E(\mathcal{F} \mid A, D) = \sum_g \sum_m w_{g,m}(A) \cdot \varepsilon_{g,m}\big(\mathcal{F}_{g,m} \mid D\big)$.
- The four error mechanisms: dephasing, relaxation, stray coupling, pulse distortion.
- Results: 16 trainable weights, $\sim 40,000$ error components, $\sim 6,500$ training benchmarks.
- Healing and stitching procedures.

The paper does *not* publish: the inner-loop optimizer, weight training procedure, or specific collision-avoidance rules.

III.B. Reconstructed Error Model

III.B.1. The Four Error Components

Dephasing. Flux noise at the operating point causes frequency fluctuations proportional to the flux sensitivity:

$$\varepsilon_{\text{deph}}(f_i) = |d\frac{\omega_i}{d}\Phi|_{f_i} \cdot \sqrt{A_\Phi}$$

where A_Φ is the $1/f$ flux noise amplitude. At the sweet spot ($f = f_{\max}$), the first derivative vanishes and T_φ is maximized. *Optimization pressure*: bias idle frequencies toward $f_{\max}$.

Relaxation. Energy decay due to TLS defects and circuit coupling:

$$\varepsilon_{\text{relax}}(f_i) = \frac{1}{T_1}(f_i)$$

where $T_1(f)$ is the measured relaxation spectrum. *Optimization pressure*: avoid T_1 hotspots.

Stray coupling. Residual ZZ interaction at non-zero detuning [13]:

$$\zeta_{ij} = \frac{2(\alpha_i + \alpha_j)J_{ij}^2}{(\Delta_{ij} + \alpha_i)(\Delta_{ij} - \alpha_j)}$$

where $\alpha \sim -200$ MHz, $J \sim 5$–20 MHz, $\Delta = f_i - f_j$. Measured values: ~ 22 kHz (nearest neighbor), ~ 2.5 kHz (distance 3). *Optimization pressure*: disperse frequencies.

Pulse distortion. Control errors from finite-bandwidth electronics:

$$\varepsilon_{\text{dist}}(f_i, f_{ij}) = c_{\text{dist}} \cdot |f_{ij} - f_i|$$

Optimization pressure: minimize frequency excursion between idle and interaction points; favor symmetric CZ resonances.

III.B.2. The 16-Weight Structure

The reduction from $\sim 40,000$ components to 16 weights exploits two symmetries:

1. **Homogeneity**: all gates of the same *type* share one weight.
2. **Translation invariance**: weights are position-independent on the chip.

On a 2D lattice with CZXEB benchmarking, there are approximately 4 gate contexts (single-qubit isolated, single-qubit adjacent to blue-layer CZ, blue-layer CZ, green-layer CZ) $\times$ 4 mechanisms $= 16$ weights:

Context	Deph.	Relax.	Stray	Dist.
SQ (isolated)	w_1	w_2	w_3	w_4
SQ (adj. blue CZ)	w_5	w_6	w_7	w_8
CZ (blue layer)	w_9	w_{10}	w_{11}	w_{12}
CZ (green layer)	w_{13}	w_{14}	w_{15}	w_{16}

Table 8: Plausible decomposition of the 16 trainable weights into 4 gate contexts $\times$ 4 error mechanisms. The physical error components $\varepsilon_{g,m}$ are computed from characterization data; only the relative importances $w_{g,m}$ are learned.

III.B.3. Weight Training Procedure

We propose training via constrained least-squares regression:

1. Acquire CZXEB cycle errors $e_{c,k}$ at $K \sim 6,500$ diverse frequency configurations $\mathcal{F}_k$.
2. For each configuration, compute the predicted error $E(\mathcal{F}_k \mid \boldsymbol{w}, D)$ using the error model.
3. Minimize $\sum_{k=1}^{K} \left[E(\mathcal{F}_k \mid \boldsymbol{w}, D) - e_{c,k}\right]^2$ over the 16 non-negative weights $\boldsymbol{w}$.
4. The frequency tunability is key: sweeping frequencies generates training data at different operating points with different error balances, breaking degeneracies.

The model evaluates at ~ 100 Hz on a desktop [12], making L-BFGS-B with non-negativity constraints computationally trivial.

III.C. Proposed Inner-Loop Optimizers

III.C.1. Candidate 1: Nelder-Mead Simplex (Most Likely)

Google uses Nelder-Mead for CZ gate optimization with 4–12 parameters per gate [14]. For $S = 2$ (default scope), each subproblem has ~ 5 dimensions—well within Nelder-Mead's effective range ($\leq$ 10–20 dimensions).

Advantages: derivative-free, handles noisy objectives, simple to implement, fast convergence for low-dimensional problems. **Disadvantages**: no convergence guarantees, can stall in high dimensions.

III.C.2. Candidate 2: CMA-ES (Best for Higher Scope)

Google has used CMA-ES for pulse optimization with up to 55 parameters [15]. For $S = 4$ (~ 21 dimensions), CMA-ES provides robust exploration with noise resistance.

Advantages: handles 10–100 dimensions, adapts covariance matrix to problem geometry, superior noise resistance (ranked best across all scenarios by Pack *et al.* [16]). **Disadvantages**: higher computational cost per iteration.

III.C.3. Candidate 3: Grid Search (For S = 1)

At $S = 1$ (1D subproblems, ~ 100 discrete options), exhaustive grid search completes in ~ 6 seconds total for 177 subproblems at 100 Hz evaluation rate. This likely explains the $S = 1$ runtime.

III.C.4. Recommended Strategy

Use **grid search** for $S = 1$, **Nelder-Mead** for $S = 2$–3, and **CMA-ES** for $S \geq 4$. This matches the published runtimes (6 s, 130 s, 6,500 s) and is consistent with Google's demonstrated optimizer preferences.

III.D. Frequency Collision Rules

For frequency-tunable transmons (Google architecture), the critical collision conditions are [17]:

Collision	Condition	Type
$\|01\angle \leftrightarrow \|10\angle$	$f_{01}^i = f_{01}^j$	Resonant swap
$\|11\angle \leftrightarrow \|02\angle$	$f_{01}^i = f_{01}^j + \alpha_j$	CZ target / parasitic
$\|11\angle \leftrightarrow \|20\angle$	$f_{01}^i = f_{01}^j - \alpha_i$	Parasitic
$\|01\angle \leftrightarrow \|12\angle$	$f_{01}^i = f_{12}^j$	Leakage
$\|10\angle \leftrightarrow \|21\angle$	$f_{01}^j = f_{12}^i$	Leakage

Table 9: Frequency collision conditions for tunable transmons. Non-target pairs must satisfy $|f_i - f_j - \alpha_j| \gg J$ (coupling strength) for all collision types.

For fixed-frequency transmons (IBM architecture), nine constraint families with specific MHz thresholds have been published [18], mapping the problem to constrained graph coloring solvable by mixed-integer programming.

III.E. Reconstructed Pseudocode

```
def snake_optimize(graph, scope_S, char_data, weights):
    F = initial_checkerboard_config(graph)
    calibrated = set()
    for subgoal in partition_graph(graph, scope_S):  # parallel
        seed = select_seed(subgoal)
        snake_traverse(seed, subgoal, F, calibrated, scope_S,
                       char_data, weights)
    return F

def snake_traverse(node, subgoal, F, calibrated, S, D, w):
    if node in calibrated: return
    local = get_neighborhood(node, S)  # ~S^2 frequencies
    constraints = {F[n] for n in calibrated if adjacent(n, local)}
    def objective(f_local):
        F_trial = F.copy(); F_trial.update(local, f_local)
        return error_estimator(F_trial, w, D)
    if S == 1:
        f_opt = grid_search(objective, 100 points)
    elif S <= 3:
        f_opt = nelder_mead(objective, F[local], constraints)
    else:
        f_opt = cma_es(objective, F[local], sigma0=50 MHz)
    F.update(local, f_opt); calibrated.add(node)
    for neighbor in traversal_options(node, S, subgoal, calibrated):
        snake_traverse(neighbor, subgoal, F, calibrated, S, D, w)
```

Listing 2: Reconstructed Snake optimizer pseudocode. The key insight is that intermediate scope ($S = 2$–4) outperforms both local ($S = 1$) and global ($S = S_{\max}$) because constraint propagation guides subsequent optimizations.

IV. Gap 5: TLS Forecasting Algorithms

IV.A. The TLS Problem

Two-level system (TLS) defects couple to transmon qubits, causing frequency-dependent relaxation (T_1) degradation. A single resonant TLS reduces T_1 by $\sim 60\%$ and increases single-qubit error by $35\times$ [19]. Google's Willow paper [20] states: *"we employ a frequency optimization strategy which forecasts defect frequencies of two-level systems (TLS)"* — but provides no algorithmic details.

IV.B. Key Physical Parameters

Our research established the quantitative foundation for TLS forecasting:

Parameter	**Value**	**Source**
Spectral diffusion constant D	2.2 ± 0.1 MHz/h1/2	[21]
TLS drift rate	Several MHz/day at 10 mK	Multiple
Slow switching rates	0.07–1.9 mHz	[22]
Fast switching rates	Up to 10 Hz	[9]
TLS density (junction leads)	0.4–0.7 / GHz / μm	[21]
Coupling threshold (strong)	$g/2\pi \geq 0.5$ MHz	[23]
Cosmic ray scrambling rate	~ 1 / 592 s	[24]
T_1 impact (single resonant TLS)	60% reduction	[19]

Table 10: Key quantitative parameters for TLS forecasting. The spectral diffusion constant $D = 2.2$ MHz/h1/2 is the central number for any prediction algorithm.

IV.C. Proposed Algorithms

IV.C.1. Algorithm 1: Bayesian Particle Filter (Physics-Motivated)

Model each TLS as a particle diffusing in frequency space:

$$f_{\mathrm{TLS},i}(t+\delta t) = f_{\mathrm{TLS},i}(t) + \sqrt{2D\cdot\delta t}\cdot\xi, \quad \xi \sim \mathcal{N}(0,1)$$

with telegraphic activation/deactivation:

$$P(\text{switch}) = r_i \cdot \delta t, \quad r_i \in [10^{-4}, 10] \text{ Hz}$$

Observation model (relaxation rate as function of qubit frequency):

$$\Gamma_1(f_q, t) = \Gamma_{1,\mathrm{bg}}(f_q) + \sum_i \frac{g_i^2 \gamma_i}{\gamma_i^2 + \left(f_q - f_{\mathrm{TLS},i}(t)\right)^2} \cdot \mathrm{active}_{i(t)}$$

Inference: Sequential Monte Carlo with $N \sim 1000$–$10,000$ particles per TLS; resample when ESS $< N/2$; spawn new TLS on unexplained T_1 dips.

Prediction horizon: $\sigma_{f(\tau)} = \sqrt{2D\tau}$ gives $\sim$ 3–5 MHz at 1–4 hours, $\sim$ 10–15 MHz at 24 hours.

Computational cost: $\mathcal{O}\big(N_{\text{particles}} \cdot N_{\text{TLS}}\big)$ per update, $\sim$ ms on CPU.

IV.C.2. Algorithm 2: Hidden Markov Model

Discretize the frequency range (e.g., 4–6 GHz) into ~ 2000 bins of width 1 MHz. Model TLS occupation as binary per bin with Gaussian transition kernels. Forward-backward inference; $\mathcal{O}(N_{\text{bins}})$ per update with sparse transitions.

IV.C.3. Algorithm 3: Neural Network Time-Series Forecaster

Train LSTM/Transformer on rolling 48-hour windows of T_1 measurements (10-minute resolution = 288 time steps). Input: per-qubit T_1 history + operating frequency + cross-qubit correlations. Output: predicted T_1 distribution over next 1–24 hours.

Requirement: $> 10,000$ qubit-hours of training data. Achieves best short-term accuracy but is device-specific and black-box.

IV.C.4. Algorithm 4: Gaussian Process Regression

Model $T_1(f, t)$ as a GP with a physics-informed composite kernel:

$$k((f_1, t_1), (f_2, t_2)) = k_{\text{freq}}(f_1, f_2) \cdot k_{\text{time}}(t_1, t_2) + k_{\text{TLS}}$$

where k_{freq} is Matérn with lengthscale $\sim$ TLS linewidth (0.1–1 MHz), k_{time} combines RBF (slow drift) and Matérn (fast switching), and k_{TLS} is a spectral mixture kernel. Provides natural uncertainty quantification.

IV.C.5. Algorithm 5: Change-Point Detection + Linear Extrapolation (Most Likely Google Approach)

This is our best reconstruction of Google's actual method, based on all available evidence:

Phase 1: Catalog. Swap spectroscopy at cooldown; record $\{f_{\text{TLS}}, g, \gamma\}$ for each defect.

Phase 2: Track. Monitor T_1 at operating frequency (every 4 QEC rounds). Periodic T_1-vs.-frequency rescans every 2–4 hours.

Phase 3: Forecast. Per tracked TLS, maintain running frequency estimate and drift velocity. Linear extrapolation with diffusion uncertainty:

$$f_{\text{TLS}}(t + \tau) \sim \mathcal{N}\Big(\hat{f}_{\text{TLS}}(t) + \hat{v}\tau, 2D\tau + \sigma^2_{\text{meas}}\Big)$$

Phase 4: Reallocate. When TLS forecast to enter operating band (within g/π), trigger recalibration via Snake optimizer.

Why this is likely Google's approach: (1) Google has swap spectroscopy capability (all papers since 2018); (2) the Willow paper uses "forecasts" (plural), suggesting per-TLS tracking; (3) they acknowledge failure for "transient TLS moving faster than our forecasts"—consistent with

diffusion-based prediction failing for telegraphic events; (4) scales linearly with qubit count; (5) integrates naturally with the Snake optimizer's relaxation-rate spectra.

IV.C.6. Algorithm 6: FPGA-Based Real-Time Bayesian Tracker (State-of-Art 2026)

Based on Berritta *et al.* [9]: gamma-distribution Bayesian estimation on FPGA. Each T_1 update takes ~ 2.2 μs; full estimate from 50 shots in ~ 11 ms. Resolves TLS switching at up to 10 Hz—4 orders of magnitude faster than previously possible.

Extension for forecasting: run parallel trackers at multiple frequency points (via AC Stark shift); detect TLS as transient Lorentzian dips in $\Gamma_1(f)$; track dip center over time; predict via diffusion model.

IV.D. Comparison of Algorithms

Algorithm	**Accuracy (1–4 h)**	**Data req.**	**Compute**	**Scalability**
Particle filter	3–5 MHz	Moderate	ms/update	Good
HMM	3–5 MHz	Moderate	ms/update	Good
Neural network	Best	Extensive	ms (GPU)	Device-specific
GP regression	3–5 MHz	Moderate	s/update	Moderate
Change-point + extrap.	5–10 MHz	Minimal	μs	Excellent
FPGA Bayesian	Sub-MHz	Real-time	μs (FPGA)	Excellent

Table 11: Comparison of proposed TLS forecasting algorithms. Accuracy column shows predicted frequency uncertainty at a 1–4 hour horizon, given $D = 2.2$ MHz/h1/2.

IV.E. Fundamental Limitations

Two processes are *inherently unpredictable*:

1. **Telegraphic TLS switching** (> 1 Hz): Can only be detected after the fact; the best strategy is fast detection + rapid recalibration [9].
2. **Cosmic ray TLS scrambling** (~ 1 event / 10 min): Simultaneously shifts multiple TLS frequencies; fundamentally stochastic [24]. Mitigation: detect via correlated multi-qubit error bursts, then re-characterize.

V. Gap 10: Control Electronics Stack

V.A. Target Specifications (Willow)

Parameter	Willow Value
1Q gate fidelity	99.97% (error 3×10^{-4})
2Q gate fidelity	99.88% (error 1.2×10^{-3})
Readout fidelity	99.5% (error 5×10^{-3})
T_1 (mean)	68 μs
1Q gate time	25 ns
2Q gate time (CZ)	42 ns
Qubit frequency range	5–7 GHz
Qubits	105

Table 12: Willow processor target specifications that the electronics stack must support.

V.B. Minimum Electronics Requirements

Parameter	Requirement	Rationale
DAC resolution	$\geq$ 14-bit	Quantization noise below 10^{-4} gate error
DAC sample rate	$\geq$ 1 GSa/s (IQ) or $\geq$ 2 GSa/s (direct)	Nyquist for 25 ns pulses
Phase noise (10 kHz offset)	< -110 dBc/Hz	Dephasing contribution $< 10^{-3}$
Output noise density	< -135 dBm/Hz	Below decoherence floor
Timing jitter	$<$ 1 ps RMS	Coherent multi-qubit sequences
Bandwidth	$\geq$ 500 MHz	DRAG-shaped 25 ns pulses
Feedback latency	$<$ 500 ns	Active reset / real-time decoding
DC flux noise	< 20 nV/$\sqrt{\text{Hz}}$	Flux bias stability

Table 13: Minimum electronics specifications for Willow-level gate fidelity. At this fidelity level, qubit decoherence (T_1, T_2) dominates over electronics noise.

V.C. Commercial Platform Comparison

Spec	**ZI SHFQC+**	**QM OPX1000**	**Qblox**	**Keysight**	**QICK**
DAC rate	2 GSa/s	2 GSa/s	1 GSa/s	$\sim$ 11 GSa/s	9.85 GSa/s
DAC bits	14	16	16	12	14
Freq. range	DC–8.5 GHz	DC–10.5 GHz	2–18.5 GHz	DC–16 GHz	DC–10 GHz
BW	1 GHz	800 MHz	500 MHz	2 GHz	$\sim$ 500 MHz
Phase noise[a]	-110	-125	-115	—	-70[b]
Feedback	350 ns	$<$ 160 ns	364 ns	—	—
Ch / unit	6+16	64	80	$\sim$ 16	16
Open source	No	No	No	No	**Yes**
Est. cost	\$150–300k	\$200–500k	\$100–250k	\$100–200k	**\$30–50k**

Table 14: Commercial quantum control electronics comparison. [a]dBc/Hz at 10 kHz offset from $\sim$ 5 GHz carrier. [b]QICK phase noise is worst-case measured; typical operation is better.

All five commercial platforms meet the minimum specifications for Willow-level 1Q and 2Q gate fidelity. The performance ceiling at this fidelity level is set by **qubit coherence and fabrication quality, not electronics noise**.

V.C.1. QICK: The Open-Source Option

QICK (Quantum Instrumentation Control Kit) [25] from Fermilab/U.
Chicago runs on Xilinx RFSoC boards:

- **ZCU216**: 16 DAC channels at 9.85 GSa/s, 14-bit, mixer-free direct RF synthesis to 10 GHz. Cost: $\sim$\$30–50k.
- **RFSoC 4x2**: 2 DAC channels. Cost: $\sim$\$2,149 (academic).
- **Demonstrated fidelity**: 99.93% (1Q Clifford), $>$ 99.9% (2Q parametric).
- Fully open-source (MIT license).

For a 105-qubit system with $\sim$ 3 lines/qubit ($\sim$ 315 channels), $\sim$ 20 ZCU216 boards would be needed ($\sim$ \$600k–\$1M), plus DC sources for flux bias.

V.D. Wiring and Filtering Scheme

V.D.1. XY Drive Lines (Qubit Control, 4–8 GHz)

Temperature Stage	Component	Purpose
300 K	SMA to semi-rigid coax	Signal routing
50 K	0–10 dB attenuator	Thermalization
4 K	10–20 dB attenuator + CuNi coax	Thermal noise suppression
$\sim$ 800 mK (still)	0–3 dB attenuator	Additional filtering
$\sim$ 100 mK (cold plate)	10–20 dB attenuator	Thermal noise
$\sim$ 10 mK (MXC)	20 dB atten. + Eccosorb + LP filter	Quantum noise floor

Table 15: XY drive line wiring scheme. Total attenuation: $\sim$ 60 dB, achieving thermal photon number $\sim 10^{-3}$ at the mixing chamber.

V.D.2. Z Flux Lines (DC–500 MHz)

Resistive coax (CuNi/stainless steel) from 300 K to 4 K; NbTi or Cu loom to MXC; RC or copper-powder filter at MXC (LP cutoff $\sim$ 1 GHz).

V.D.3. Readout Output Lines (4–8 GHz)

- **MXC (10 mK)**: TWPA or JPA ($\geq$ 20 dB gain, near quantum limit) + 2–3 circulators/isolators.
- **4 K**: HEMT amplifier ($\sim$ 40 dB gain, $\sim$ 2–4 K noise temperature).
- **300 K**: Room-temperature amplification chain + digitization.
- Cable: NbTi superconducting coax (MXC to 4 K, ultra-low loss), stainless steel above.

V.E. Scaling Beyond 100 Qubits

At 1000+ qubits, room-temperature electronics face fundamental limits:

- **Wiring**: $\sim$ 3000–5000 coaxial lines through the cryostat; high-density solutions (Bluefors Cri/oFlex, flexible striplines) become essential.
- **Thermal budget**: Each coax conducts $\sim$ 20–50 μW from 300 K to 4 K; MXC cooling power is only $\sim$ 10–20 μW.
- **Cost**: $\sim$ \$5–50M in control hardware at commercial pricing.

The path forward is **cryogenic CMOS**: controllers operating at 4 K, demonstrated by both Google [26] and IBM, achieving gate errors of 10^{-4}–10^{-3} at 4–23 mW per qubit.

VI. Synthesis and Recommendations

VI.A. Feasibility Assessment

Gap	Solution Maturity	Key Remaining Challenge	Effort
1	High — 52-node DAG reconstructed from 5 sources	Tolerance fine-tuning requires iterative experimental validation	3–6 months
3	High — error model fully reconstructed; 3 candidate inner-loop optimizers	Weight training requires $\sim 6,500$ CZXEB benchmarks on actual hardware	2–4 months
5	Medium — 6 algorithms proposed; fundamental prediction limits quantified	Algorithm 5 (change-point) needs experimental validation of D on specific device	4–8 months
10	High — 5 commercial + 1 open-source platform meet specifications	Wiring/filtering at 100+ qubits requires cryogenic engineering	1–3 months

Table 16: Feasibility assessment for bridging each critical gap. Effort estimates assume access to a multi-qubit transmon testbed.

VI.B. Recommended Reproduction Roadmap

Months 1–3: Deploy control electronics (QICK on ZCU216 or OPX1000); implement the 52-node calibration DAG in Qibocal or QUAlibrate; bring up a 5–10 qubit testbed.

Months 4–6: Implement TLS characterization (swap spectroscopy, T_1 monitoring); begin building TLS catalog; implement Algorithm 5 (change-point + extrapolation) as baseline forecaster.

Months 7–9: Implement the Snake outer loop with Nelder-Mead inner loop; collect CZXEB training data; train the 16-weight error model.

Months 10–12: Scale to 20+ qubits; validate TLS forecasting accuracy; implement healing procedure for outlier suppression; compare forecasting algorithms (particle filter vs. change-point).

Year 2: Scale to 50–100 qubits; implement stitching for large-scale optimization; deploy FPGA-based real-time T_1 tracking; target below-threshold QEC.

References

[1] J. Kelly, P. O'Malley, M. Neeley, H. Neven, and J. M. Martinis, Physical qubit calibration on a directed acyclic graph, Arxiv Preprint arXiv:1803.03226 (2018).

[2] A. Pasquale and others, Qibocal: an open-source framework for calibration of self-hosted quantum devices, Arxiv Preprint arXiv:2410.00101 (2024).

[3] Quantum Machines, *Qualibrate: Open-Source Calibration Framework*, https://github.com/qua-platform/qualibrate.

[4] IBM Research, *Qiskit Experiments: Characterization and Calibration Library*, https://qiskit-community.github.io/qiskit-experiments/.

[5] Zurich Instruments, *Labone Q: Automated Qubit Bring-up*, https://www.zhinst.com/europe/en/blogs/automated-qubit-bring-labone-q.

[6] J. S. Kelly, *Automatic Qubit Calibration*, US9940212B2 (2018).

[7] M. A. Marciniak and others, Millisecond-Scale Calibration and Benchmarking of Superconducting Qubits, Arxiv Preprint arXiv:2602.11912 (2026).

[8] A. Bengtsson, A. Opremcak, M. Khezri, and others, Model-based Optimization of Superconducting Qubit Readout, Physical Review Letters **132**, arXiv:2308.02079 (2024).

[9] F. Berritta and others, Real-time adaptive tracking of fluctuating relaxation rates in superconducting qubits, Arxiv Preprint arXiv:2506.09576 (2026).

[10] others, Flux distortion calibration at the sub-permille level for superconducting qubit gates, Arxiv Preprint arXiv:2503.04610 (2025).

[11] others, Characterization of tunable couplers in superconducting qubit systems, Arxiv Preprint arXiv:2501.13646 (2025).

[12] P. V. Klimov, A. Bengtsson, C. Quintana, and others, Optimizing quantum gates towards the scale of logical qubits, Nature Communications **15**, 10.1038/s41467-024-46623-y (2024).

[13] others, Crosstalk dispersion and spatial scaling in superconducting qubit arrays, Arxiv Preprint arXiv:2512.18148 (2025).

[14] D. Fan and others, Calibrating quantum gates up to 52 qubits in a superconducting processor, Npj Quantum Information **11**, arXiv:2505.22390 (2025).

[15] others, Leakage reduction in fast superconducting qubit gates via optimal control, Npj Quantum Information 10.1038/s41534-020-00346-2 (2020).

[16] K. Pack, S. Machnes, and F. K. Wilhelm, Benchmarking Optimization Algorithms for Automated Calibration of Quantum Devices, Arxiv Preprint arXiv:2509.08555 (2024).

[17] others, Analysis of Frequency Collisions in Parametrically Modulated Circuits, Arxiv Preprint arXiv:2511.05031 (2025).

[18] others, Efficient Frequency Allocation for Superconducting Quantum Processors, Arxiv Preprint arXiv:2410.20283 (2024).

[19] others, Scalable site-specific TLS frequency tuning in superconducting qubits, Arxiv Preprint arXiv:2503.04702 (2025).

[20] Google Quantum AI, Quantum error correction below the surface code threshold, Nature 10.1038/s41586-024-08449-y (2024).

[21] others, Statistics of strongly coupled defects in superconducting qubits, Arxiv Preprint arXiv:2506.00193 (2025).

[22] P. V. Klimov and others, Fluctuations of Energy-Relaxation Times in Superconducting Qubits, Physical Review Letters **121**, 10.1103/PhysRevLett.121.090502 (2018).

[23] others, Mitigating losses from strongly coupled defect modes in superconducting qubits, Arxiv Preprint arXiv:2407.18746 (2024).

[24] others, Cosmic rays and correlated errors in superconducting qubit arrays, Nature Communications 10.1038/s41467-025-61385-x (2025).

[25] L. Stefanazzi and others, QICK: Quantum Instrumentation Control Kit, Review of Scientific Instruments **93**, arXiv:2110.00557 (2022).

[26] IBM Research, *A Cryo-CMOS Control System for Large-Scale Superconducting Qubit Quantum Computing*, https://research.ibm.com/publications/.